

icfes: competencia: - formulacion\_ejecucion nivel\_dificultad: 3 contenido: categoria: geometria tipo: no\_generico contexto: matematico eje\_axial: eje2 componente: geometrico\_metrico

```
options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorizamos los nombres para contextualizar el problema
nombres <- c("Camilo", "Andrés", "Sofía", "Manuel", "Laura", "Carlos",
             "Daniela", "Miguel", "Valentina", "Eduardo", "Natalia",
             "José", "Isabella", "Gabriel", "Mariana", "Santiago", "Lucía",
             "Alejandro", "Catalina", "Mateo", "Valeria", "Sebastián", "Juliana")
nombre <- sample(nombres, 1)

# Aleatorizamos los tipos de recipientes
recipientes <- c("cilindro", "tubo", "conducto", "tanque cilíndrico",
                "recipiente cilíndrico", "contenedor cilíndrico",
                "ducto cilíndrico", "depósito cilíndrico", "cañería cilíndrica")
recipiente <- sample(recipientes, 1)

# Aleatorizar adjetivos para el cilindro interno
adjetivos_interno <- c("interno", "hueco", "vacío", "interior", "central", "medio")
adjetivo_interno <- sample(adjetivos_interno, 1)

# Aleatorizar líquidos
liquidos <- c("aceite", "combustible", "líquido", "fluido", "agua", "refrigerante",
             "solución", "mezcla", "sustancia")
liquido <- sample(liquidos, 1)

# Aleatorizar lo que se desea calcular
calculos <- c("la cantidad de", "el volumen de", "cuánto", "qué cantidad de",
             "cuántos litros de", "qué volumen de", "la capacidad de")
calculo <- sample(calculos, 1)

# Aleatorizar verbos para la acción
verbos <- c("llenar", "contener", "almacenar", "ocupar", "requerir")
verbo <- sample(verbos, 1)

# Aleatorizar medidas del cilindro (manteniendo consistencia matemática)
# Primero generamos el radio interno (entre 0.1 y 0.4 metros)
r_int <- round(runif(1, 0.1, 0.4), 2)

# Luego generamos el radio externo, asegurando que sea mayor que el interno
# Radio externo entre 1.5 y 2.5 veces el radio interno
factor_radio <- runif(1, 1.5, 2.5)
r_ext <- round(r_int * factor_radio, 2)

# Calculamos el diámetro externo (exactamente 2 veces el radio externo)
d_ext <- 2 * r_ext

# Altura del cilindro (entre 0.5 y 4 metros)
altura <- round(runif(1, 0.5, 4), 2)
```

```

# Calcular el grosor de la pared del cilindro
grosor <- r_ext - r_int

# Verificar coherencia física
if (grosor < 0.05) {
  # Asegurar un grosor mínimo de 0.05 unidades
  grosor <- round(runif(1, 0.05, 0.2), 2)
  r_ext <- r_int + grosor
  # Asegurar que el diámetro externo sea exactamente 2 veces el radio externo
  d_ext <- 2 * r_ext
}

# Calcular el volumen necesario (es el volumen del cilindro interior)
volumen <- round(pi * (r_int^2) * altura, 2)

# Aleatorizar unidades de medida
unidades_longitud <- c("m", "cm", "dm")
unidad <- sample(unidades_longitud, 1)

# Ajustar valores según la unidad
factor_conversion <- 1
if (unidad == "cm") {
  factor_conversion <- 100
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
} else if (unidad == "dm") {
  factor_conversion <- 10
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
}

# Redondear todos los valores para mayor claridad
d_ext <- round(d_ext, 1)
r_int <- round(r_int, 1)
altura <- round(altura, 1)
r_ext <- round(r_ext, 1)
volumen <- round(volumen, 1)

# La respuesta correcta es "C) Altura del cilindro"
# Ya que para calcular el volumen necesitaríamos conocer la altura
opcion_correcta <- 3 # Índice de "Altura del cilindro" en el vector de opciones

# Vector de solución para r-exams (1 para la correcta, 0 para las incorrectas)
solucion <- c(0, 0, 1, 0)

```

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

```

```

# Función para formatear valores numéricos (1 decimal para decimales, 0 para enteros)

```

```

formatear_numero <- function(numero) {
  if (numero %% 1 == 0) {
    return(as.integer(numero))
  } else {
    return(sprintf("%.1f", numero))
  }
}

# Formatear los valores
d_ext_fmt <- formatear_numero(d_ext)
r_ext_fmt <- formatear_numero(r_ext)
r_int_fmt <- formatear_numero(r_int)
altura_fmt <- formatear_numero(altura)
grosor_fmt <- formatear_numero(grosor)

# Código Python para generar el cilindro hueco similar a la imagen de referencia
codigo_python <- paste0("
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Ellipse, FancyArrowPatch, Polygon

# Parámetros del cilindro
radiointerno = ", r_int, " # ", unidad, "
radioexterno = ", r_ext, " # ", unidad, "
altura = ", altura, " # ", unidad, "
diametro_externo = ", d_ext, " # ", unidad, "
grosor = radioexterno - radiointerno # ", unidad, "
unidad = '"', unidad, '"

# Crear figura
fig, ax = plt.subplots(figsize=(8, 8))

# Colores
azul_cilindro = '#4682B4' # Azul para contornos
azul_relleno = '#EBF5FF' # Azul muy claro para el relleno
gris_linea = '#777777' # Gris para líneas punteadas

# Factores de perspectiva para las elipses
factor_elipse = 0.35
vradio_ext = radioexterno * factor_elipse
vradio_int = radiointerno * factor_elipse

# Centros del cilindro
centro_superior = (0, altura)
centro_inferior = (0, 0)

# Rellenar el cilindro con color azul claro
# Crear coordenadas para el cuerpo exterior visible
theta_vals = np.linspace(-np.pi, 0, 50)
x_ext_front = radioexterno * np.cos(theta_vals)
y_ext_front_inf = vradio_ext * np.sin(theta_vals)
y_ext_front_sup = altura + vradio_ext * np.sin(theta_vals)

```

```

# Polígono para el cuerpo exterior
ext_body_points = []
for i in range(len(theta_vals)):
    ext_body_points.append((x_ext_front[i], y_ext_front_inf[i]))
ext_body_points.append((radioexterno, 0))
ext_body_points.append((radioexterno, altura))
for i in range(len(theta_vals)-1, -1, -1):
    ext_body_points.append((x_ext_front[i], y_ext_front_sup[i]))
ext_body_polygon = Polygon(ext_body_points, closed=True, facecolor=azul_relleno,
                           edgecolor='none', alpha=0.5, zorder=1)
ax.add_patch(ext_body_polygon)

# Polígono para el cuerpo interior (hueco)
x_int_front = radiointerno * np.cos(theta_vals)
y_int_front_inf = vradio_int * np.sin(theta_vals)
y_int_front_sup = altura + vradio_int * np.sin(theta_vals)

int_body_points = []
for i in range(len(theta_vals)):
    int_body_points.append((x_int_front[i], y_int_front_inf[i]))
int_body_points.append((radiointerno, 0))
int_body_points.append((radiointerno, altura))
for i in range(len(theta_vals)-1, -1, -1):
    int_body_points.append((x_int_front[i], y_int_front_sup[i]))
int_body_polygon = Polygon(int_body_points, closed=True, facecolor='white',
                           edgecolor='none', zorder=2)
ax.add_patch(int_body_polygon)

# Dibujar líneas verticales
ax.plot([radioexterno, radioexterno], [0, altura], color=azul_cilindro, linewidth=1.5, zorder=5)
ax.plot([-radioexterno, -radioexterno], [0, altura], color=azul_cilindro, linewidth=1.5, zorder=5)
ax.plot([radiointerno, radiointerno], [0, altura], color=azul_cilindro, linewidth=1.5, zorder=5)
ax.plot([-radiointerno, -radiointerno], [0, altura], color=azul_cilindro, linewidth=1.5, zorder=5)

# LÍNEAS PUNTEADAS PARA LAS PARTES NO VISIBLES DE LAS TAPAS SUPERIOR E INFERIOR
# Semiélipses posteriores (arco trasero)
theta_back = np.linspace(0, np.pi, 100) # Más puntos para curvas más suaves
x_back_ext = radioexterno * np.cos(theta_back)
y_back_ext_inf = vradio_ext * np.sin(theta_back)
y_back_ext_sup = altura + vradio_ext * np.sin(theta_back)
x_back_int = radiointerno * np.cos(theta_back)
y_back_int_inf = vradio_int * np.sin(theta_back)
y_back_int_sup = altura + vradio_int * np.sin(theta_back)

# Dibujar líneas punteadas para las semiélipses posteriores
linea_punteada = {'linestyle': '--', 'color': gris_linea, 'linewidth': 1.2, 'zorder': 3,
                  'dashes': [3, 2]} # Líneas punteadas correctas

# Semiélipses posteriores inferiores
ax.plot(x_back_ext, y_back_ext_inf, **linea_punteada)
ax.plot(x_back_int, y_back_int_inf, **linea_punteada)

# Semiélipses posteriores superiores

```

```

ax.plot(x_back_ext, y_back_ext_sup, **linea_punteada)
ax.plot(x_back_int, y_back_int_sup, **linea_punteada)

# Dibujar semielipses frontales visibles
# Semiellipses frontales inferiores
theta_front = np.linspace(-np.pi, 0, 100)
x_front_ext = radioexterno * np.cos(theta_front)
y_front_ext_inf = vradio_ext * np.sin(theta_front)
x_front_int = radiointerno * np.cos(theta_front)
y_front_int_inf = vradio_int * np.sin(theta_front)

ax.plot(x_front_ext, y_front_ext_inf, color=azul_cilindro, linewidth=1.5, zorder=5)
ax.plot(x_front_int, y_front_int_inf, color=azul_cilindro, linewidth=1.5, zorder=5)

# Semiellipses frontales superiores
y_front_ext_sup = altura + vradio_ext * np.sin(theta_front)
y_front_int_sup = altura + vradio_int * np.sin(theta_front)
ax.plot(x_front_ext, y_front_ext_sup, color=azul_cilindro, linewidth=1.5, zorder=5)
ax.plot(x_front_int, y_front_int_sup, color=azul_cilindro, linewidth=1.5, zorder=5)

# Añadir puntos centrales
ax.plot(0, altura, 'o', color=azul_cilindro, markersize=4, zorder=6)
ax.plot(0, 0, 'o', color=azul_cilindro, markersize=4, zorder=6)

# Añadir flechas de radio
ax.add_patch(FancyArrowPatch((0, altura), (radiointerno, altura), arrowstyle='->',
                             color=azul_cilindro, linewidth=1, mutation_scale=10, zorder=6))
ax.add_patch(FancyArrowPatch((0, 0), (radioexterno, 0), arrowstyle='->',
                             color=azul_cilindro, linewidth=1, mutation_scale=10, zorder=6))

# FLECHAS Y ETIQUETAS DE DIMENSIONES
# Propiedades comunes para flechas
flecha_props = dict(arrowstyle='<->', color='black', linewidth=1, mutation_scale=10)

# 1. DIÁMETRO EXTERNO (ajustado para mejor visibilidad)
diam_y = altura + vradio_ext*2 + altura*0.25
ax.add_patch(FancyArrowPatch((-radioexterno, diam_y), (radioexterno, diam_y), **flecha_props))
ax.text(0, diam_y + altura*0.1, f'Diámetro externo = {"", d_ext_fmt, "} {unidad}',
        fontweight='bold', ha='center', va='center', fontsize=9)

# 2. ALTURA (ajustado para mejor espacio)
altura_x = radioexterno + radioexterno*0.5
ax.add_patch(FancyArrowPatch((altura_x, 0), (altura_x, altura), **flecha_props))
ax.text(altura_x + radioexterno*0.15, altura/2, 'Altura',
        fontweight='bold', rotation=90, ha='center', va='center', fontsize=9)

# 3. RADIO INTERNO (ajustado para mejor posición)
radio_int_label_pos = (radiointerno*2.5, altura + altura*0.3)
ax.plot([radiointerno, radio_int_label_pos[0]], [altura, radio_int_label_pos[1]], 'k-', linewidth=0.8)
ax.plot([radio_int_label_pos[0]-0.1, radio_int_label_pos[0]], [radio_int_label_pos[1], radio_int_label_pos[1]], 'k-', linewidth=0.8)
ax.text(radio_int_label_pos[0], radio_int_label_pos[1],
        f'Radio interno = {"", r_int_fmt, "} {unidad}',
        fontweight='bold', ha='left', va='center', fontsize=9)

```

```

# 4. GROSOR (ajustado para mejor posición)
grosor_start = -radioexterno + (radioexterno - radiointerno)/2
ax.add_patch(FancyArrowPatch((-radiointerno, altura), (-radioexterno, altura), **flecha_props, zorder=6))
grosor_label_pos = (-radioexterno*2.5, altura + altura*0.25)
ax.plot([-radioexterno, grosor_label_pos[0]], [altura, grosor_label_pos[1]], 'k-', linewidth=0.8)
ax.plot([grosor_label_pos[0], grosor_label_pos[0]+0.1], [grosor_label_pos[1], grosor_label_pos[1]], 'k-')
ax.text(grosor_label_pos[0], grosor_label_pos[1],
        f'Grosor = {"", grosor_fmt, ""} {unidad}',
        fontweight='bold', ha='right', va='center', fontsize=9)

# 5. RADIO EXTERNO (ajustado para mejor visibilidad)
radio_ext_label_pos = (radioexterno*2.5, -altura*0.3)
ax.plot([radioexterno, radio_ext_label_pos[0]], [0, radio_ext_label_pos[1]], 'k-', linewidth=0.8)
ax.plot([radio_ext_label_pos[0]-0.1, radio_ext_label_pos[0]], [radio_ext_label_pos[1], radio_ext_label_pos[1]], 'k-')
ax.text(radio_ext_label_pos[0], radio_ext_label_pos[1],
        f'Radio externo = {"", r_ext_fmt, ""} {unidad}',
        fontweight='bold', ha='left', va='center', fontsize=9)

# Ajustar límites y aspecto de la figura (ampliado para incluir todas las etiquetas)
margin = max(radioexterno, altura) * 0.9
ax.set_xlim(-radioexterno - margin, radioexterno + margin)
ax.set_ylim(-margin*0.8, altura + margin*1.2)
ax.set_aspect('equal')
ax.axis('off')

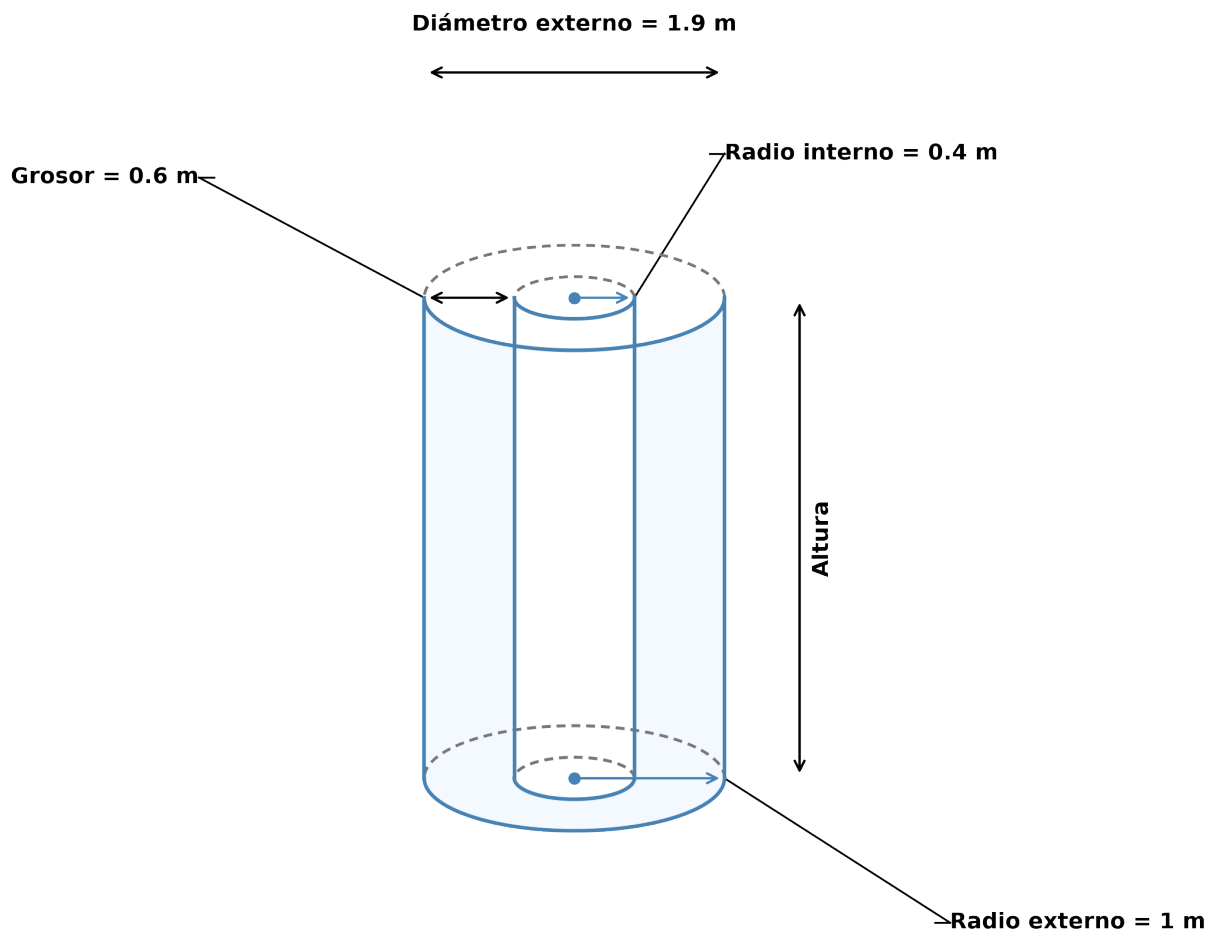
# Guardar la figura con más espacio para etiquetas
plt.tight_layout(pad=1.5)
plt.savefig('cilindro_hueco.png', dpi=300, bbox_inches='tight', transparent=True)
plt.close()
")

# Ejecutar código Python para generar la figura
py_run_string(codigo_python)

```

## Question

Natalia desea saber cuánto(a) combustible se necesita para llenar un cañería cilíndrica interno, pero solamente cuenta con las medidas de las dimensiones que muestra la figura.



| Dimensión        | Valor | Unidad |
|------------------|-------|--------|
| Radio interno    | 0,4   | m      |
| Radio externo    | 1     | m      |
| Diámetro externo | 1,9   | m      |
| Grosor           | 0,6   | m      |

¿Cuál medida le falta a Natalia para hallar la cantidad deseada?

## Answerlist

- Radio externo.
- Diámetro externo.
- Altura del cilindro.
- Perímetro del cilindro.

## Solution

La respuesta correcta es: **Altura del cilindro.**

Para calcular el volumen de combustible necesario para llenar el cañería cilíndrica central, necesitamos calcular el volumen de un cilindro, cuya fórmula es:

$$V = \pi \cdot r^2 \cdot h$$

Donde:

- $r$  es el radio (en este caso, el radio interno = 0,4 m)
- $h$  es la altura del cilindro

En el problema se nos proporciona:

- Diámetro externo = 1,9 m
- Radio interno = 0,4 m

Sin embargo, no se nos proporciona la altura del cilindro, que es esencial para calcular el volumen. Por lo tanto, a Natalia le falta conocer la altura del cilindro para poder calcular la cantidad de combustible necesaria.

Si tuviéramos la altura, el volumen se calcularía así:

$$V = \pi \cdot (0,4)^2 \cdot h = 0,5 \cdot h \text{ m}^3$$

## Answerlist

- Falso
- Falso
- Verdadero
- Falso

## Meta-information

exname: volumen\_cilindro\_hueco extype: schoice exsolution: 0010 exshuffle: TRUE exsection: Geometría|Volumen|Cilindro